

Next Generation Data Center Operations for SAP Enterprise Software

Large Language Models Operations (LLMOps)

Noah Kim



Curriculum version

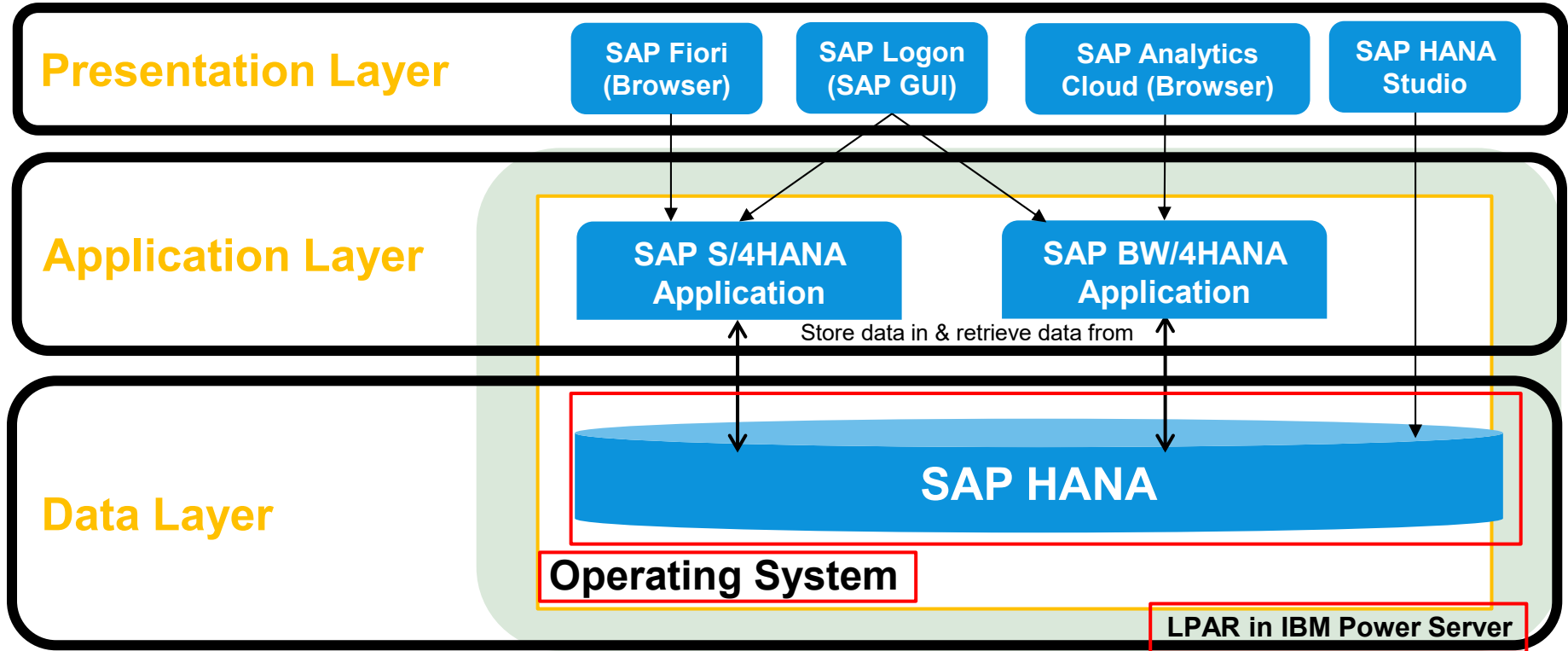
- Curriculum Version:
 - 2026
- Software used:
 - MobaXterm (only for Windows users, Mac users simply use the Terminal app)
 - AnythingLLM
 - Visual Studio Code
- Prerequisites
 - [Download eduVPN](#)
 - [Download AnythingLLM](#)



Learning objectives

- Instead of theory-heavy frontal teaching of LLMs, we familiarize you with the practice of installing and working with LLMs.

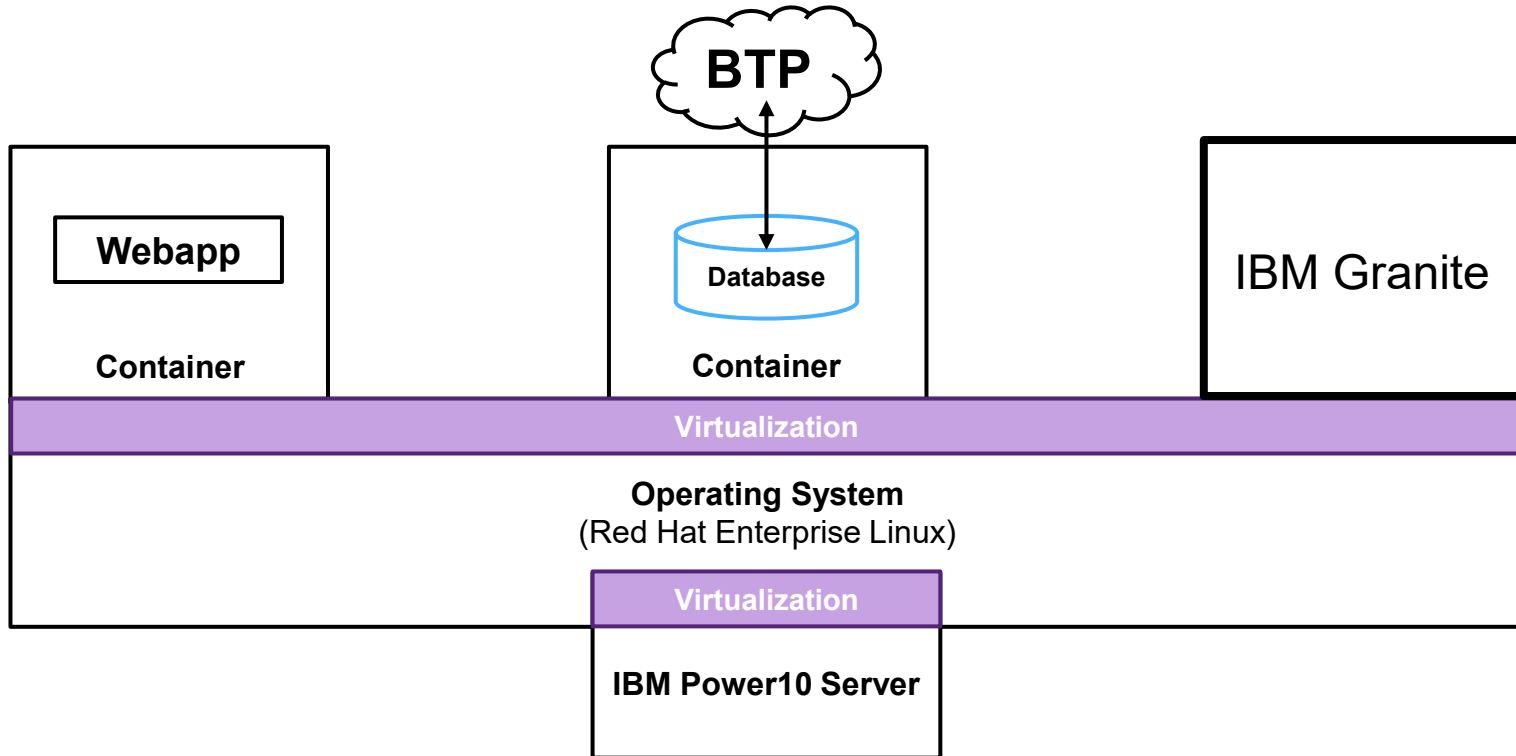
Recap



„Technical Foundations of SAP HANA“

Image Source: Self-created

Curriculum Structure



Source: Self-created

Recap

- In the last session, each of you installed an own SAP HANA Express database in their own container.
- Today, each of you will install an own LLM in their own container.

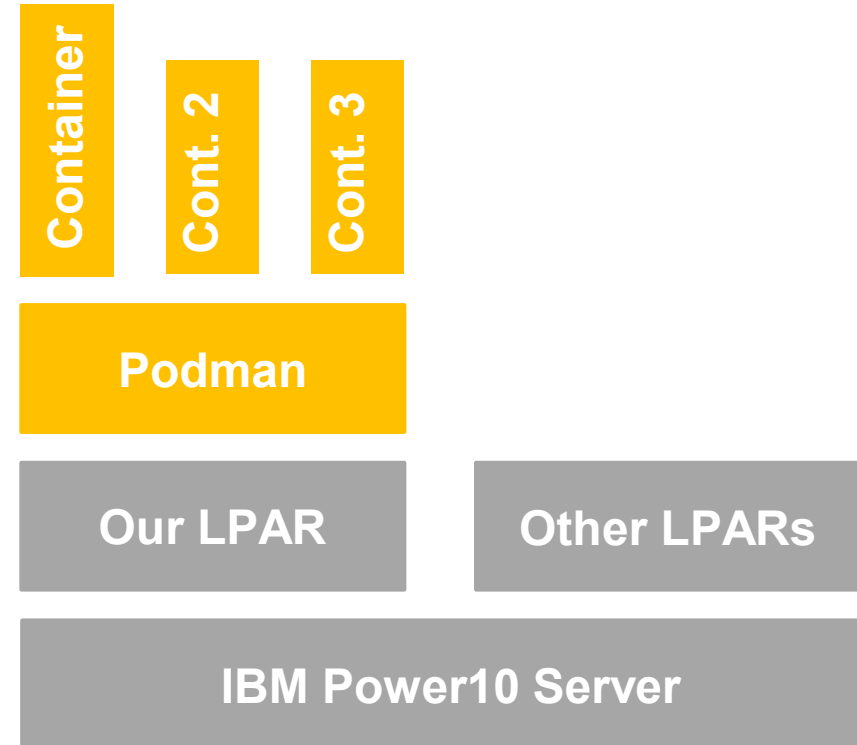


Image Source: Self-created

Recap

- Information:
 - Text file defining instructions for reproducibly building a container image
 - Includes installation and application configuration steps
- Command-Line Interface:
 - `podman build -t <image-name> .`
 - Builds a new image based on the containerfile in the current directory
 - Podman automatically recognizes containerfiles when they are named *Dockerfile* or *Containerfile*

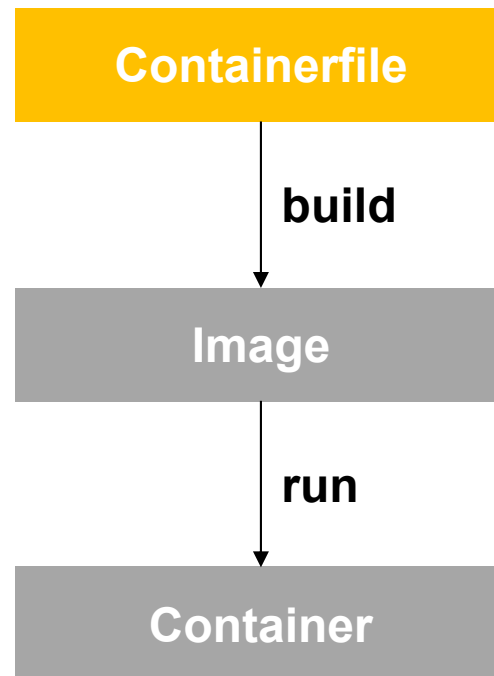
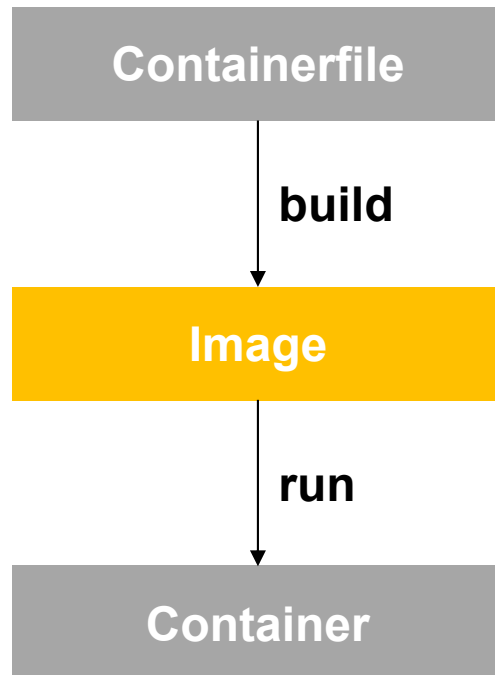


Image Source: Self-created

Recap

- Information:
 - Foundation for creating and running containers
 - Snapshot of an application and its dependencies, packaged in a portable format
- Command-Line Interface:
 - `podman images`
 - Lists available container images
 - `podman run -it <image-name> /bin/bash`
 - Creates a new container based on a specified image and starts an interactive shell session



Recap

- Information:
 - Lightweight environment encapsulating an application along with its dependencies
- Command-Line Interface:
 - `podman ps -a`
 - Lists all running and stopped containers
 - `podman start (stop) <container-id>`
 - Starts (stops) an existing container
 - `podman exec -it <container-id> /bin/bash`
 - Executes an interactive shell session for a container
 - `podman commit <container-id> <image-name>`
 - Creates a new image based on the current container state
 - `podman rm <container-id>`
 - Removes a stopped container

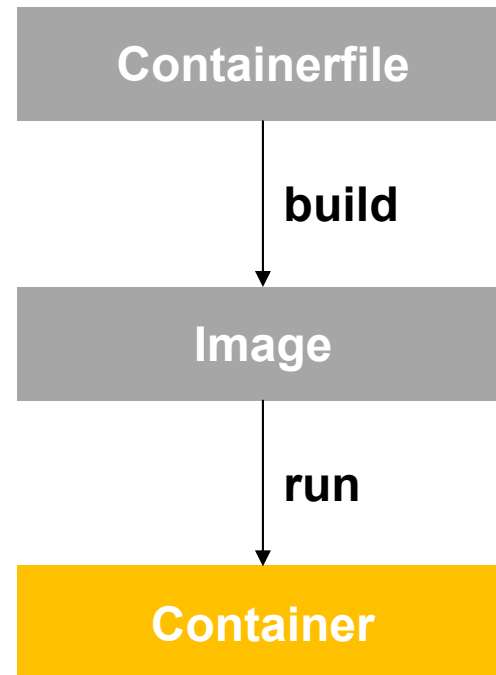


Image Source: Self-created

Recap

- What we will be doing next:
 - As in the last session, we provided an LPAR on our IBM Power server for you.
 - Again, you will connect to the LPAR using SSH.
 - You will all connect to the same LPAR, using different OS names (see info sheet).

- Make sure that you have installed [EduVPN](#) and that it is currently running.

Terms of Use

- Users must not:
 - Engage in activities that disrupt system performance and availability.
 - **Attempt unauthorized access to other systems** or data.
 - Execute malicious code, hacking attempts, or security breaches.
 - Use the system for personal, commercial, or illegal activities.

- Intentional misuse or violation of these terms may result in:
 - Immediate revocation of access rights.
 - Reporting of the incident to the respective academic institution.
 - Potential legal consequences in cases of severe misconduct.

Terms of Use

- We may use the monitored data to evaluate our teaching process, and may publish the results.
- However, the OS names are anonymized and we will not publish any personal information about you.
- **We encourage you to participate**, and if you are not doing well, that is fine.
Your inputs will not be graded. 😊
- **By accessing the LPAR, you automatically agreed to all Terms of Use.**

Connecting to the Linux LPAR (Win)

- Open MobaXTerm and set a master password.
 - You can choose this password yourself, but you **need to remember it!**
- Go to *Session* → *SSH*
 - Remote host: *<HOSTNAME>*
 - Check-mark the *Specify username* option
 - Username: *<YOUR LEARN-ID>*
 - Port 22
- Click on *OK*

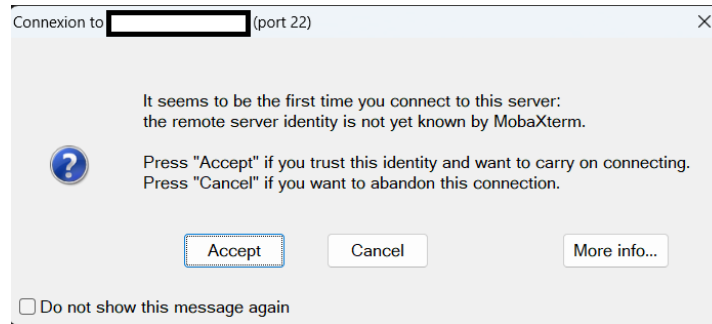
Connection Details

- Please write down your connection details:
- Host URL (for everyone): **SoSe26DC.ucc.cit.tum.de**

Name	User	Password	Port
Group 1	granite1	granite123	5001
Group 2	granite2	granite123	5002
Group 3	granite3	granite123	5003
Group 4	granite4	granite123	5004
Group 5	granite5	granite123	5005
Group 6	granite6	granite123	5006
Group 7	granite7	granite123	5007
Group 8	granite8	granite123	5008
Group 9	granite9	granite123	5009

Image Source: Self-created

Connecting to the Linux LPAR (Win)



Important note:

If you see this pop-up, then you are able to access the machine 😊 Click on Accept.

If not, then you are not connected to the same network where our LPAR is located.

You need to use the EduVPN (TUM).

Connecting to the Linux LPAR (Mac)

- Open the terminal and type: **ssh** <USER>@<HOSTNAME>
 - For example, if your <User> is granite-student1 and the hostname is workshop.ucc.cit.tum.de:
e.g. *ssh granite1@sose26dc.ucc.cit.tum.de*
- There should appear a message:
 - *The authenticity of host <HOSTNAME> can't be established*
 - ...
 - *Are you sure you want to continue connecting (yes/no/[fingerprint])?*
- **If you see this message, then you are able to access the machine 😊**
 - Type yes and hit the *Enter* key
 - **If not**, then you are not connected to the same network where our LPAR is located.
You need to use the EduVPN (TUM).

Logging-In

- Use the password from the information table we provided for you (*SSH Connection*)
- Note: Type carefully as the terminal will not display any of your password! Use CTRL+u to reset your input to nothing.
- **Raise your hand if problems occur when logging-in.**

Recap

- Some basics:
 - **ls** command:
 - List the contents of a directory, simply using **ls** lists the contents of the **current** directory
 - ***Try it out now, type ls and hit Enter.***
 - **cd** command:
 - Change the directory, e.g. **cd <name_of_directory>**
 - Use **cd ..** to move up one directory level

Recap

- Some basics:
 - **nano <file-name>** to edit a file
 - **CTRL+O**, **Enter** and **CTRL+X** to save the file and leave nano
 - **chmod +x <file-name>.sh** to make a script executable
 - **./<script-name>.sh** to execute a script

Agenda

- Recap
- **Basics**
- Deploying IBM Granite on Power10
- Zero-Shot Experiments
- Retrieval-Augmented Generation (RAG)
- Function Calling (Agentic AI)
- Visual Studio Code + Continue
- Autocomplete

Why LLMs? Why local?

- LLM stands for Large Language Model: AI systems trained on billions of parameters to understand and generate human-like text for tasks like chatbots, code generation, document analysis, and customer support

Why local?

- Advantages:
 - Data stays on your hardware, data privacy compliant
 - Fully customizable: Adapt LLM to your business case (e.g. Fine-Tuning)
- Disadvantage:
 - Hardware cost and maintenance
 - Expertise needed

<https://www.ibm.com/think/topics/large-language-models>

LLM Operations

- LLMOps stands for “[large language model](#) operations” and refers to the specialized practices and workflows that speed development, deployment and management of [AI models](#) throughout their complete lifecycle. (IBM)

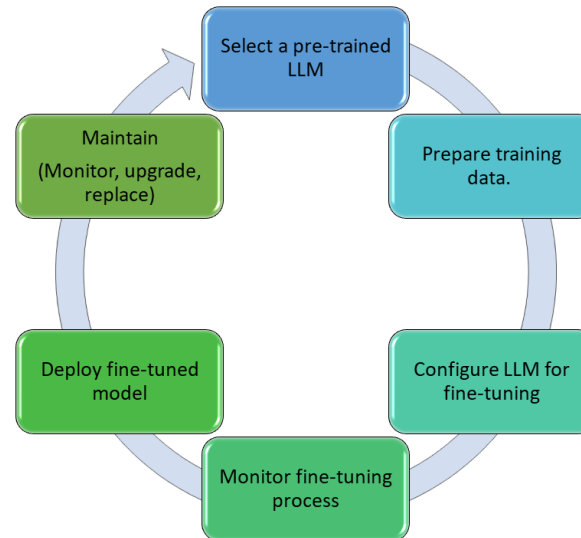


Image Source: [Granite code](#)

Introduction to IBM Granite

- IBM Granite is a family of open-source AI models ([Link](#)), e.g. **Granite for Code** ([Link](#))
 - This model was trained across 12 languages and 116 programming languages (**PRE-TRAINING**)
- Paper: Granite 3.0 Language Models ([Link](#))
- Basic Example:
 - Question: „Write a code to find the maximum value in a list of numbers.“
 - Answer:

```
```python
def find_max(numbers):
 max_value = numbers[0]
 for num in numbers:
 if num > max_value:
 max_value = num
 return max_value
```<|endoftext|>
```

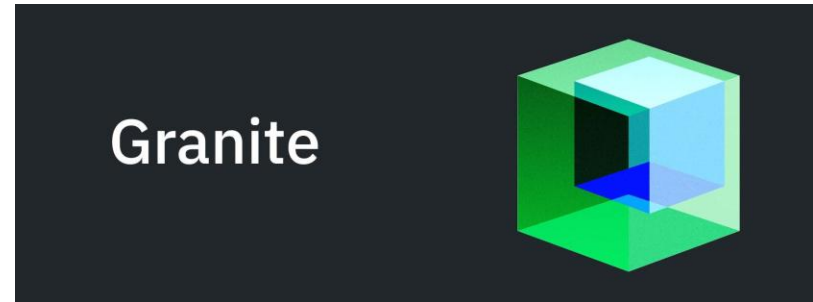


Image Source: [Granite code](#)

What is Ollama?

- Lightweight, **open-source** tool for running LLMs locally on your machine or server
- Supports models like llama3, mistral, and **GGUF-based models**, and lets you run them with simple commands
- Models can be used **offline** after download, making it suitable for privacy-sensitive environments
- Link: <https://ollama.com/>



What is GGUF?

- Check out: <https://huggingface.co/bartowski/granite-3.1-8b-instruct-GGUF>
 - After scrolling down, you see a bunch of files:
 - GGUF is a file format used for storing quantized (reduced size) versions of large language models (LLMs)
 - Break Down: granite-3.1-8b-instruct-Q5_K_L.gguf

Download a file (not the whole branch) from below:

Filename	Quant type	File Size	Split	Description
granite-3.1-8b-instruct-f16.gguf	f16	16.34GB	false	Full F16 weights.
granite-3.1-8b-instruct-Q8_0.gguf	Q8_0	8.68GB	false	Extremely high quality, generally unneeded but max available quant.
granite-3.1-8b-instruct-Q6_K_L.gguf	Q6_K_L	6.75GB	false	Uses Q8_0 for embed and output weights. Very high quality, near perfect, <i>recommended</i> .
granite-3.1-8b-instruct-Q6_K.gguf	Q6_K	6.71GB	false	Very high quality, near perfect, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_L.gguf	Q5_K_L	5.85GB	false	Uses Q8_0 for embed and output weights. High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_M.gguf	Q5_K_M	5.80GB	false	High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_S.gguf	Q5_K_S	5.65GB	false	High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q4_K_L.gguf	Q4_K_L	4.99GB	false	Uses Q8_0 for embed and output weights. Good quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q4_K_M.gguf	Q4_K_M	4.94GB	false	Good quality, default size for most use cases, <i>recommended</i> .

...

Image Source: <https://huggingface.co/bartowski/granite-3.1-8b-instruct-GGUF>

What is GGUF?

- Check out: <https://huggingface.co/bartowski/granite-3.1-8b-instruct-GGUF>
 - After scrolling down, you see a bunch of files:
 - GGUF is a file format used for storing quantized (reduced size) versions of large language models (LLMs)
 - Break Down: granite-3.1-8b-instruct-Q5_K_L.gguf

Model size, 8 billion parameters

With **more parameters**, LLMs can solve **more complex tasks**
But: It is also **more computationally expensive** to train models with a high number of parameters, thus we need to find a trade-off based on the hardware specifications

Download a file (not the whole branch) from below:

Filename	Quant type	File Size	Split	Description
granite-3.1-8b-instruct-f16.gguf	f16	16.34GB	false	Full F16 weights.
granite-3.1-8b-instruct-Q8_0.gguf	Q8_0	8.68GB	false	Extremely high quality, generally unneeded but max available quant.
granite-3.1-8b-instruct-Q6_K_L.gguf	Q6_K_L	6.75GB	false	Uses Q8_0 for embed and output weights. Very high quality, near perfect, <i>recommended</i> .
granite-3.1-8b-instruct-Q6_K.gguf	Q6_K	6.71GB	false	Very high quality, near perfect, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_L.gguf	Q5_K_L	5.85GB	false	Uses Q8_0 for embed and output weights. High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_M.gguf	Q5_K_M	5.80GB	false	High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_S.gguf	Q5_K_S	5.65GB	false	High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q4_K_L.gguf	Q4_K_L	4.99GB	false	Uses Q8_0 for embed and output weights. Good quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q4_K_M.gguf	Q4_K_M	4.94GB	false	Good quality, default size for most use cases, <i>recommended</i> .
...				

Image Source: <https://huggingface.co/bartowski/granite-3.1-8b-instruct-GGUF>

What is GGUF?

- Check out: <https://huggingface.co/bartowski/granite-3.1-8b-instruct-GGUF>
 - After scrolling down, you see a bunch of files:
 - GGUF is a file format used for storing quantized (reduced size) versions of large language models (LLMs)
 - Break Down: granite-3.1-8b-instruct-Q5_K_L.gguf

designed for executing specific
user instructions accurately and effectively

Chat: designed for conversational interactions

Download a file (not the whole branch) from below:

Filename	Quant type	File Size	Split	Description
granite-3.1-8b-instruct-f16.gguf	f16	16.34GB	false	Full F16 weights.
granite-3.1-8b-instruct-Q8_0.gguf	Q8_0	8.68GB	false	Extremely high quality, generally unneeded but max available quant.
granite-3.1-8b-instruct-Q6_K_L.gguf	Q6_K_L	6.75GB	false	Uses Q8_0 for embed and output weights. Very high quality, near perfect, <i>recommended</i> .
granite-3.1-8b-instruct-Q6_K.gguf	Q6_K	6.71GB	false	Very high quality, near perfect, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_L.gguf	Q5_K_L	5.85GB	false	Uses Q8_0 for embed and output weights. High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_M.gguf	Q5_K_M	5.80GB	false	High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_S.gguf	Q5_K_S	5.65GB	false	High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q4_K_L.gguf	Q4_K_L	4.99GB	false	Uses Q8_0 for embed and output weights. Good quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q4_K_M.gguf	Q4_K_M	4.94GB	false	Good quality, default size for most use cases, <i>recommended</i> .

...

Image Source: <https://huggingface.co/bartowski/granite-3.1-8b-instruct-GGUF>

What is GGUF?

- Check out: <https://huggingface.co/bartowski/granite-3.1-8b-instruct-GGUF>
 - After scrolling down, you see a bunch of files:
 - GGUF is a file format used for storing quantized (reduced size) versions of large language models (LLMs)
 - Break Down: granite-3.1-8b-instruct-Q5_K_L.gguf

Quantization format used to reduce the model size while maintaining performance

- Q5: 5-bit quantization
- K_L: quantization method

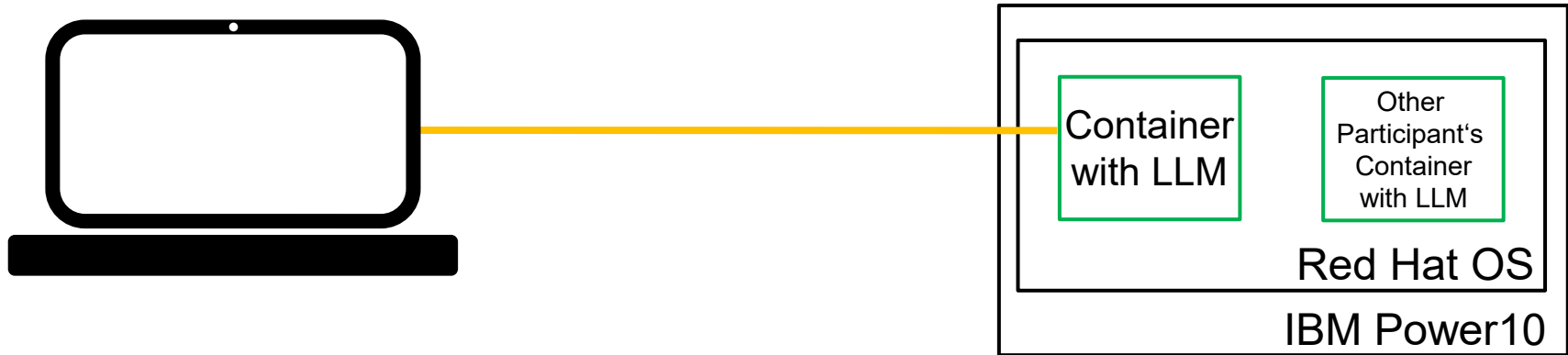
Download a file (not the whole branch) from below:

Filename	Quant type	File Size	Split	Description
granite-3.1-8b-instruct-f16.gguf	f16	16.34GB	false	Full F16 weights.
granite-3.1-8b-instruct-Q8_0.gguf	Q8_0	8.68GB	false	Extremely high quality, generally unneeded but max available quant.
granite-3.1-8b-instruct-Q6_K_L.gguf	Q6_K_L	6.75GB	false	Uses Q8_0 for embed and output weights. Very high quality, near perfect, <i>recommended</i> .
granite-3.1-8b-instruct-Q6_K.gguf	Q6_K	6.71GB	false	Very high quality, near perfect, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_L.gguf	Q5_K_L	5.85GB	false	Uses Q8_0 for embed and output weights. High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_M.gguf	Q5_K_M	5.80GB	false	High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q5_K_S.gguf	Q5_K_S	5.65GB	false	High quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q4_K_L.gguf	Q4_K_L	4.99GB	false	Uses Q8_0 for embed and output weights. Good quality, <i>recommended</i> .
granite-3.1-8b-instruct-Q4_K_M.gguf	Q4_K_M	4.94GB	false	Good quality, default size for most use cases, <i>recommended</i> .
...				

Image Source: <https://huggingface.co/bartowski/granite-3.1-8b-instruct-GGUF>

Next Steps

- Groups of 2-3 deploy their own LLM in a Container
- Each Group can use this LLM on their personal computer using AnythingLLM
 - Without SSH (connecting to the container)
 - You simulate both the **hosting** (Power10) and the **clienting** side (application on your computer)

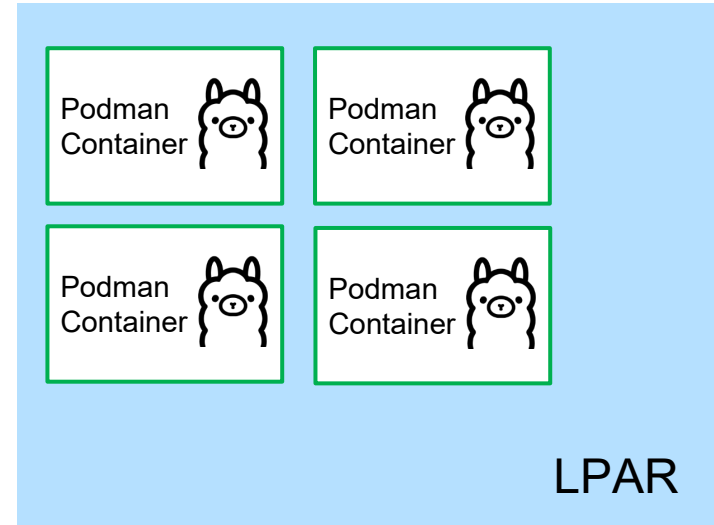


Agenda

- Recap
- Basics
- **Deploying IBM Granite on Power10**
- Zero-Shot Experiments
- Retrieval-Augmented Generation (RAG)
- Function Calling (Agentic AI)
- Visual Studio Code + Continue
- Autocomplete

Live: Deploying Ollama via Podman

- Each group will have their podman container on their user
- Each group can only see their own container of their user
- Each user download their own files



Live: Deploying Ollama via Podman

- Log into the user using ssh (MobaXterm, Terminal etc.)
- Verify with **pwd** and **ls** your current directory and files (you should be in home)
- You should see a file **create_ollama_container.sh** → read with command: **cat**

```
# Define OLLAMA_HOME directory
OLLAMA_HOME="$HOME/ollama"

# Create OLLAMA_HOME if it doesn't exist
mkdir -p "$OLLAMA_HOME"

echo "Using Ollama models directory: $OLLAMA_HOME"

echo "Starting the container with Podman..."
podman run -d \
  --name "${CONTAINER_NAME}" \
  -p ${HOST_PORT}:11434 \
  -v ${OLLAMA_HOME}:/root/.ollama:Z \
  -e OLLAMA_MAX_LOADED_MODELS=1 \
  -e OLLAMA_KEEP_ALIVE=0 \
  -e OLLAMA_NUM_PARALLEL=1 \
  "${CONTAINER_IMAGE}"

echo "Container '${CONTAINER_NAME}' started on port ${HOST_PORT}."
echo "Ollama directory stored in: $OLLAMA_HOME"
```

Podman run flags:

- d detached container
- name name of container
- p host port mapped on container port
- v volume mounted
- e environment variables

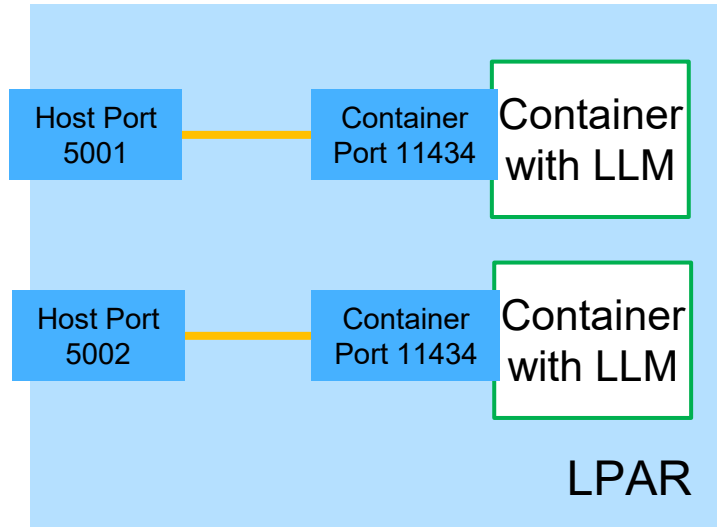
Live: Deploying Ollama via Podman

- Create the directory ollama: **mkdir ollama**
- Run the script using `./` (make sure it is executable with `chmod +x`)
- If: `-bash: ./create_ollama_container.sh: /bin/bash^M: Defekter Interpreter`
 - → `sed -i 's/\r$//' create_ollama_container.sh`
- Now enter a container name and **your** port number
Port numbers must be unique for lpar!
- The container sets up
- Use **podman ps -a** to see all containers, is it running?

```
[noah@granitelp1 ~]$ podman ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
06d7f95c639c	quay.io/mgiessing/ollama:v0.9.6	serve	20 hours ago	Up 4 minutes	0.0.0.0:5001->11434/tcp	5001
114847e4bf12	quay.io/mgiessing/ollama:v0.9.6	serve	3 minutes ago	Up 3 minutes	0.0.0.0:5002->11434/tcp	ollama

Deploying IBM Granite on Power10



```
[noah@granitelp1 ~]$ podman ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
06d7f95c639c	quay.io/mgiessing/ollama:v0.9.6	serve	20 hours ago	Up 4 minutes	0.0.0.0:5001->11434/tcp	5001
114847e4bf12	quay.io/mgiessing/ollama:v0.9.6	serve	3 minutes ago	Up 3 minutes	0.0.0.0:5002->11434/tcp	ollama

Deploying IBM Granite on Power10

- Execute an interactive shell session for the new container.

Container-ID or container name, check via **podman ps**

```
[noah@granitelp1 ~]$ podman exec -it ollama bash
root@114847e4bf12:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@114847e4bf12:/# pwd
/
root@114847e4bf12:/# exit
exit
[noah@granitelp1 ~]$
```

podman exec -it
... bash

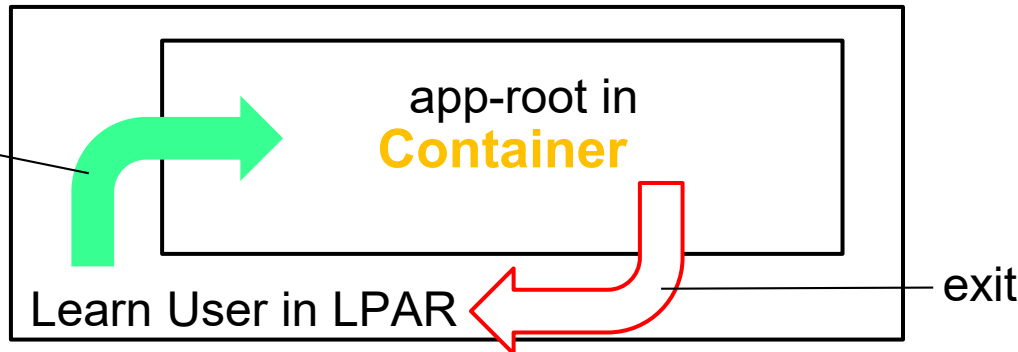


Image Source: Self-created

Deploying IBM Granite on Power10

- Enter your container as root: `podman exec -it <container_name> bash`
- Type: `ollama -h`
- Now we need a model → <https://ollama.com/library>
- Search for granite
- Type: `ollama run granite4` (run pulls the model if not in list)

```
root@114847e4bf12:/# ollama run granite4
>>> hello
Hello! How can I assist you today? If you have any questions or need information on a specific topic, feel free to ask.

>>> how yall doin
We're doing well, thanks for asking! Just here ready to help with your queries and provide useful information whenever you need it. How can I assist you today?
```

```
root@114847e4bf12:/# ollama -h
Large language model runner

Usage:
  ollama [flags]
  ollama [command]

Available Commands:
  serve      Start ollama
  create     Create a model from a Modelfile
  show       Show information for a model
  run        Run a model
  stop       Stop a running model
  pull       Pull a model from a registry
  push       Push a model to a registry
  list       List models
  ps         List running models
  cp         Copy a model
  rm         Remove a model
  help       Help about any command

Flags:
  -h, --help      help for ollama
  -v, --version    Show version information
```

Deploying IBM Granite on Power10

- More open-source models on: huggingface
- Link for granite4: <https://huggingface.co/ibm-granite/granite-4.0-micro-GGUF>
- You can run almost all GGUF without h
- Advantage: use of community based fine-tuned LLMs

Deploying IBM Granite on Power10

ibm-granite/**granite-4.0-micro-GGUF** like 20 Follow IBM Granite 4.24k

Transformers GGUF language granite-4.0 conversational License: apache-2.0

Model card Files and versions xet Community

Granite 4.0 Micro (GGUF)

This repository contains models that have been converted to the GGUF format with various quantizations from an IBM Granite base model.

Please reference the base model's full model card here: <https://huggingface.co/ibm-granite/granite-4.0-micro>

How to use from Ollama

```
ollama run hf.co/ibm-granite/granite-4.0-micro-GGUF:Q4_K_M
```

Copy

Quick Links

Read the Ollama documentation

Downloads last month
2,701

GGUF

Model size

Hardware compatibility

2-bit

3-bit

Q3_K_S | 1.57 GB

4-bit

Q4_K_S | 2 GB Q4_0 | 1.98 GB

5-bit

Q5_K_S | 2.38 GB Q5_0 | 2.38 GB

6-bit

Libraries

Transformers

llama-cpp-python

Notebooks

Google Colab

Kaggle

Local Apps

llama.cpp

LM Studio

Jan

Ollama

Unloth new

Agenda

- Recap
- Basics
- Deploying IBM Granite on Power10
- **Zero-Shot Experiments**
- Retrieval-Augmented Generation (RAG)
- Function Calling (Agentic AI)
- Visual Studio Code + Continue
- Autocomplete

Zero-Shot Experiments (Theory)

- Zero-Shot:
“Zero-shot prompting is a [prompt engineering](#) method that relies on the pretraining of a [large language model \(LLM\)](#) to infer an appropriate response. In contrast to other prompt engineering methods such as few-shot prompting, models aren’t provided examples of output when prompting with the zero-shot technique.” (IBM)
- Temperature:
„Temperature controls the randomness of text that is generated by LLMs during inference. [...] Different temperature settings essentially introduce different levels of randomness when a [generative AI](#) model outputs text.“ (IBM)
- Scale: 0 (low temperature, very deterministic) – 1 (high temperature, more random)

Zero-Shot Experiments

- Type: `ollama run granite4` (inside your container as root)
- Now you are in the chat terminal → type in `/?` for help
- Type `/set parameter` to see all parameters → set the temperature to 0
- `/bye` to exit terminal to reset

```
>>> /set parameter
Unknown command '/set parameter'. Type /? for help
>>> /set parameter
Available Parameters:
  /set parameter seed <int>           Random number seed
  /set parameter num_predict <int>    Max number of tokens to predict
  /set parameter top_k <int>         Pick from top k num of tokens
  /set parameter top_p <float>       Pick token based on sum of probabilities
  /set parameter min_p <float>       Pick token based on top token probability * min_p
  /set parameter num_ctx <int>       Set the context size
  /set parameter temperature <float> Set creativity level
  /set parameter repeat_penalty <float> How strongly to penalize repetitions
  /set parameter repeat_last_n <int> Set how far back to look for repetitions
  /set parameter num_gpu <int>       The number of layers to send to the GPU
  /set parameter stop <string> <string> ... Set the stop parameters

>>> Send a message (/? for help)
```

Zero-Shot Experiments

■ Group Task:

- Experiment with the temperature = 0, 0.5, 1.25
 - Make sure to always **/clear** to remove the context (zero-shot)
 - Ask something like: „Describe Power10 in two sentences.”
 - What do you observe?
-
- Try this with different models from the ollama website and huggingface
 - Hint: you can also: `podman exec <container> run “prompt” –verbose`
This runs the command directly into the container and gives token speed output
 - **Make sure the model size does not exceed 3GB !**

- Your Ollama container starts the serve automatically
- **serve**: starts Ollama's HTTP server that exposes all your local models via REST API on port 11434
- Container Port 11434 is mapped to your unique port (5001, 5002,...)
- For access you can test via curl on the same machine:

```
curl http://localhost:5001/api/chat -d '{  
  "model": "granite4",  
  "messages": [{  
    "role": "user",  
    "content": "Describe in two sentences what is curl?"  
  }],  
  "stream": false  
}'
```

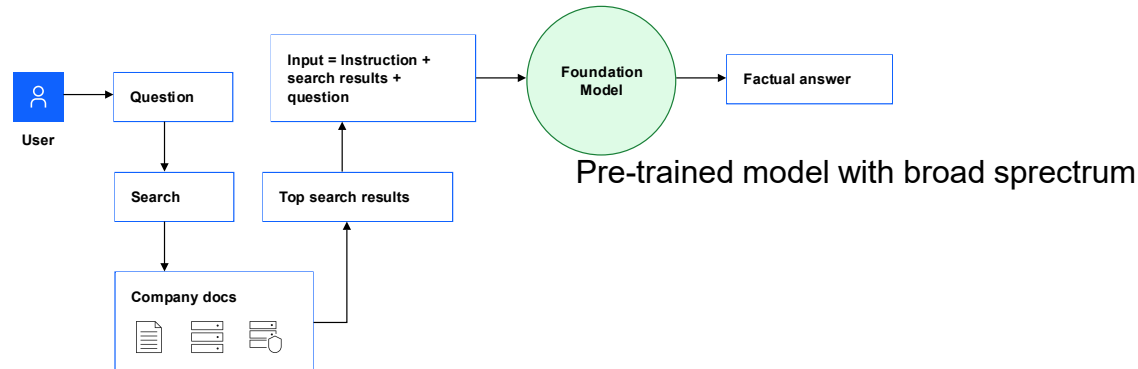
You can access it from your own terminal on your laptop, what do you need change?

Agenda

- Recap
- Basics
- Deploying IBM Granite on Power10
- Zero-Shot Experiments
- **Retrieval-Augmented Generation (RAG)**
- Function Calling (Agentic AI)
- Visual Studio Code + Continue
- Autocomplete

Retrieval-Augmented Generation

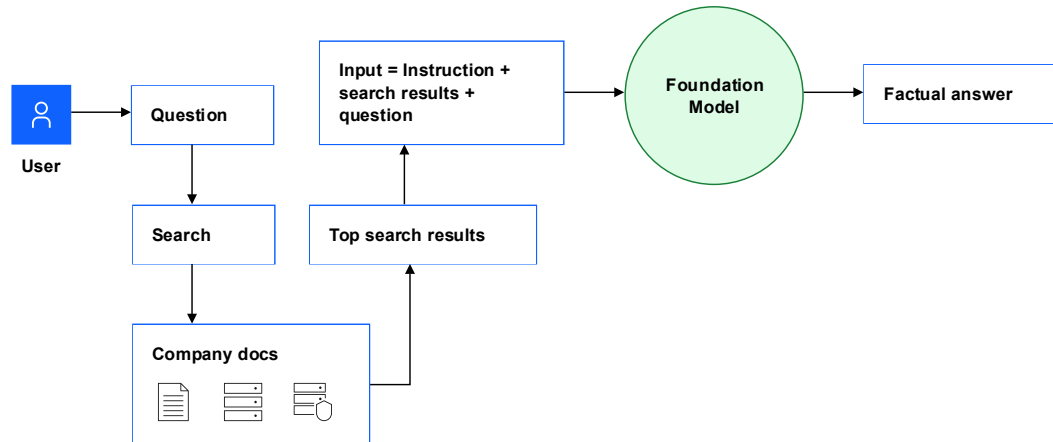
- “*Alternative*” to Fine-Tuning.
- „RAG is a cost-efficient method for **supplementing an LLM with domain-specific knowledge** that wasn’t part of its pretraining. RAG makes it possible for a chatbot to accurately answer questions related to a specific field or business **without retraining** the model. **Knowledge documents** are stored in a vector database [...]” ([Link](#))



- “Live Data”

Retrieval-Augmented Generation

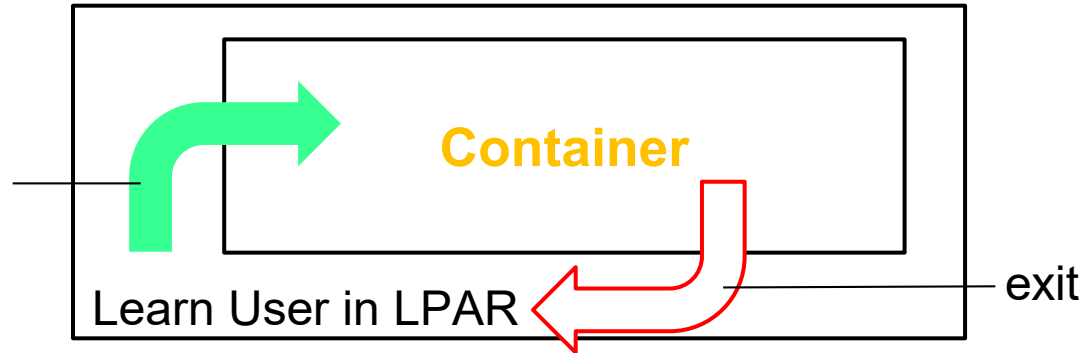
- Go to <https://anythingllm.com/> and download AnythingLLM for your OS.
- Install the application on your personal computer.



Retrieval-Augmented Generation

- During the download...
 - Go to your the LPAR with your granite user

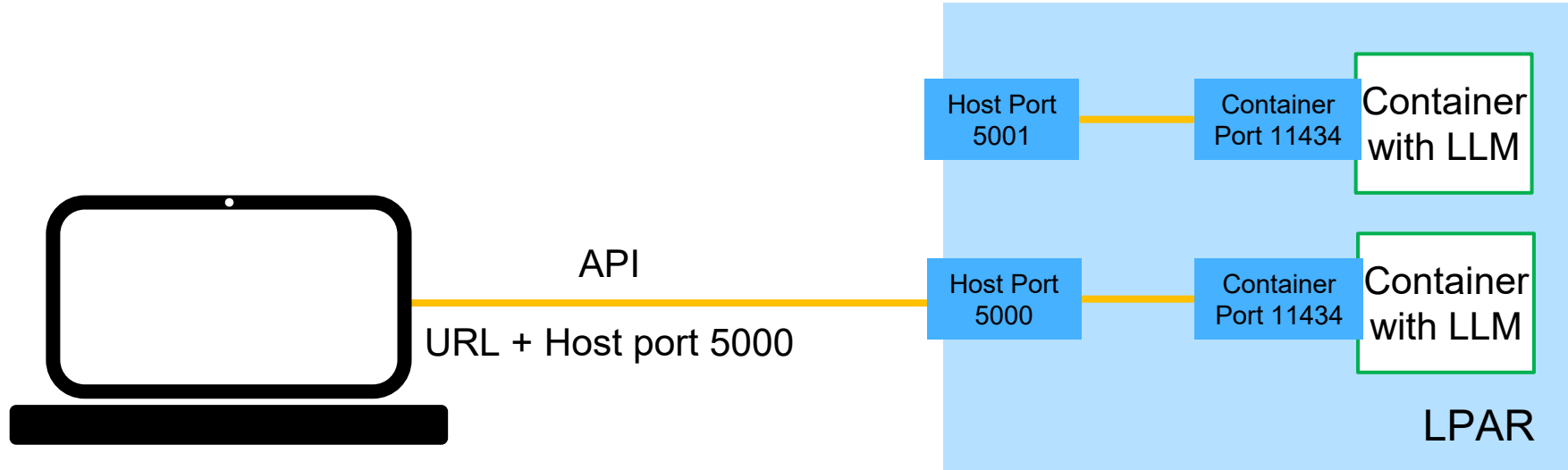
`podman exec -it
<name> bash`



- Make sure your container is running with **podman ps -a**
- Make sure you have models available with **ollama list** (inside container)

Retrieval-Augmented Generation

- Small webserver is built which is serving the LLM on Port 11434 by default



Live: AnythingLLM

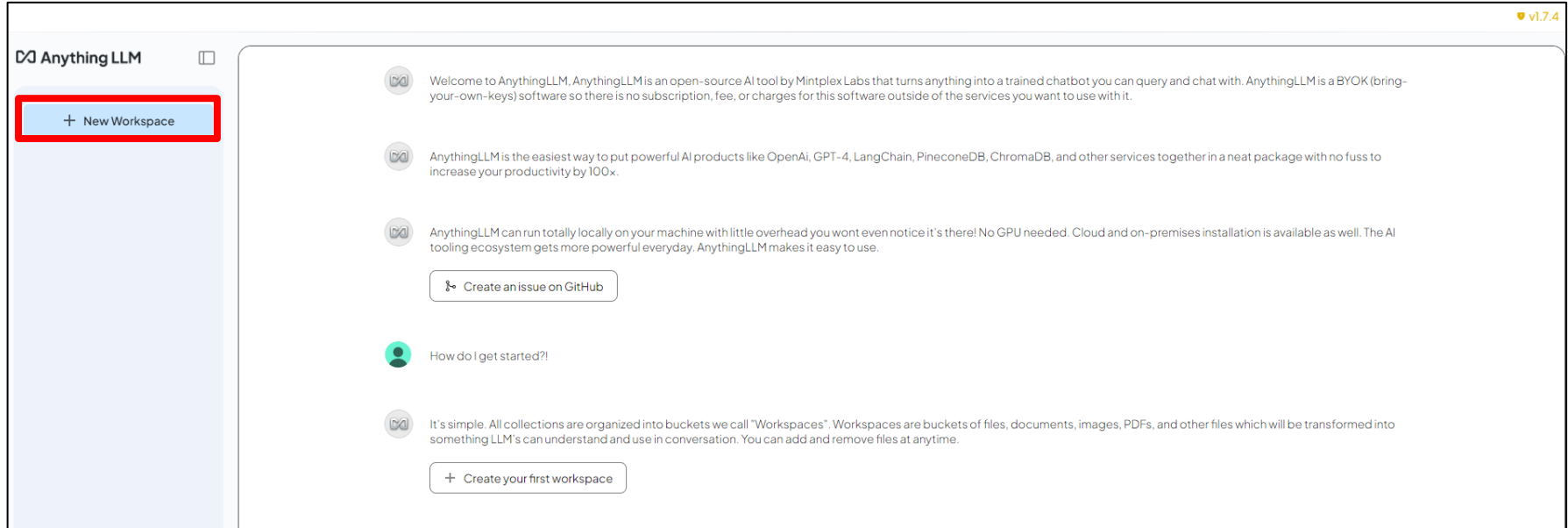


Image Source: Self-created

Live: AnythingLLM

AnythingLLM



Suchen



Mein Workspace



default

Thread



+ New Thread

Assistant Chats

Wie kann ich Ihnen heute helfen?

Schreibe eine Nachricht



Erstelle einen Agenten

Arbeitsbereich bearbeiten

Ein Dokument hochladen



Image Source: Self-created

Live: AnythingLLM

AnythingLLM

INSTANZEINSTELLUNGEN

⚙️ KI-Anbieter ▾

LLM

Vektordatenbank

Einbettung

Textsplitting & Chunking

Sprache & Sprachausgabe

👤 Administrator >

📁 Agentenfähigkeiten

🗣️ Assistent für Besprechungen

📄 Desktop-Assistent

🌐 Community Hub >

🔗 Personalisierung >

🧰 Werkzeuge >

🔬 Experimentelle Funktionen

Support kontaktieren

Datenschutz & Datenverarbeitung

LLM-Präferenz

Dies sind die Anmeldeinformationen und Einstellungen für Ihren bevorzugten LLM-Chat- und Einbettungsanbieter. Es ist wichtig, dass diese Schlüssel aktuell und korrekt sind, sonst wird AnythingLLM nicht richtig funktionieren.

LLM-Anbieter

Save changes



Ollama

Run LLMs locally on your own machine.



Ollama Model

--loading available models-- ▾

Select the Ollama model you want to use.
Models will load after entering a valid
Ollama URL.

Hide advanced settings ^

Ollama Base URL ⓘ

http://

Ollama Keep Alive ⓘ

1 hour ▾

Model context window ⓘ

Auto-detected from model

Authentication Token ⓘ

Ollama Auth Token

Image Source: Self-created

Retrieval-Augmented Generation

LLM-Anbieter

**Ollama**
Run LLMs locally on your own machine.

⌵

Ollama Model

--loading available models-- ▾

Select the Ollama model you want to use.
Models will load after entering a valid
Ollama URL.

Hide advanced settings ^

Ollama Base URL ⓘ

http://

LPAR name

<Container-Port>

Model context window ⓘ

Auto-detected from model

Ollama Keep Alive ⓘ

1 hour ▾

Authentication Token ⓘ

Ollama Auth Token

e.g., if you were assigned port 5001,
use:
`http://sose26dc.ucc.cit.tum.de:5001`
as Base URL

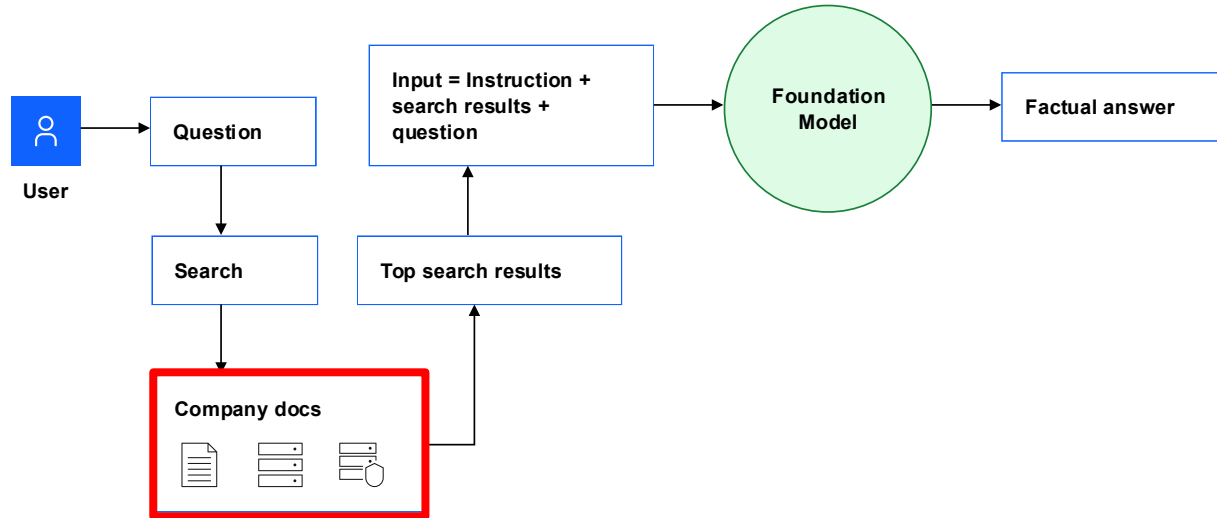
Once URL is confirmed, you can see all
available ollama model

Ollama serves all models which you pulled

Afterwards, click on
Update workspace

Source: Self-created

Retrieval-Augmented Generation

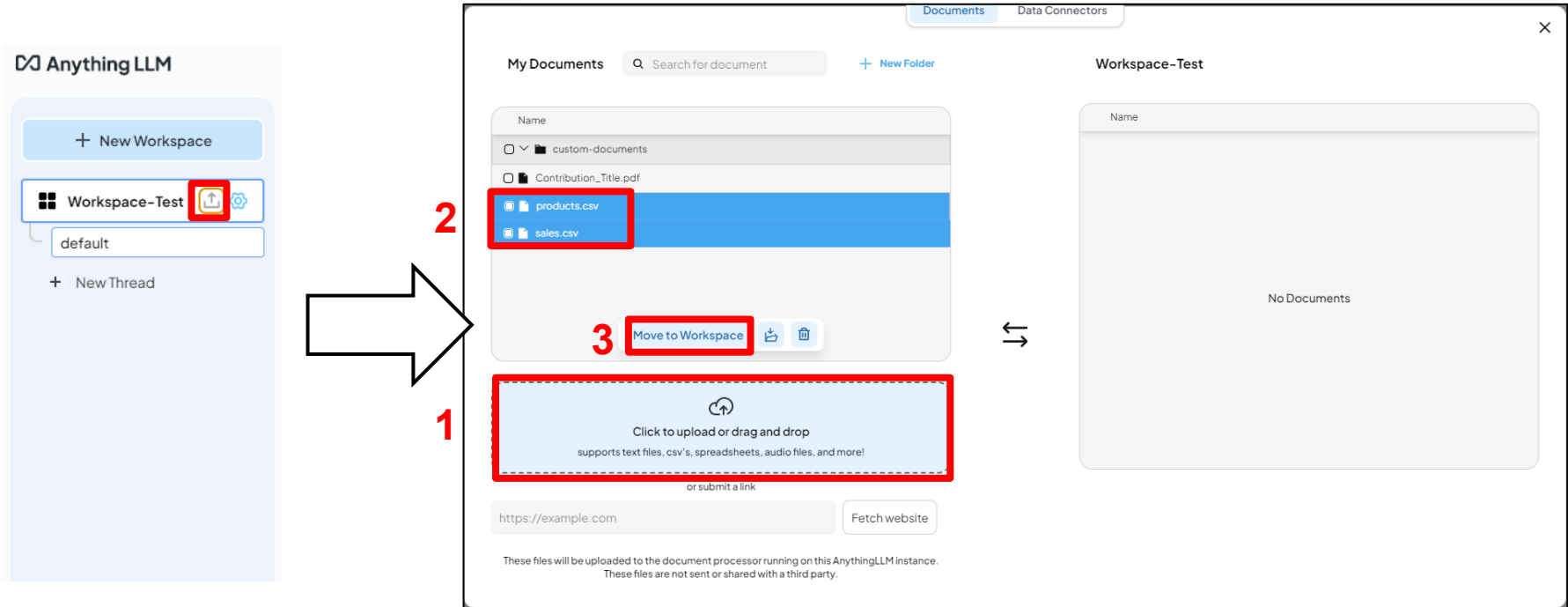


Source (Translated): [Retrieval-Augmented Generation](#)

Retrieval-Augmented Generation

- Please download the new versions of *products.csv* and *sales.csv* from the Moodle course page
- Integrate these files into your AnythingLLM workspace

Retrieval-Augmented Generation

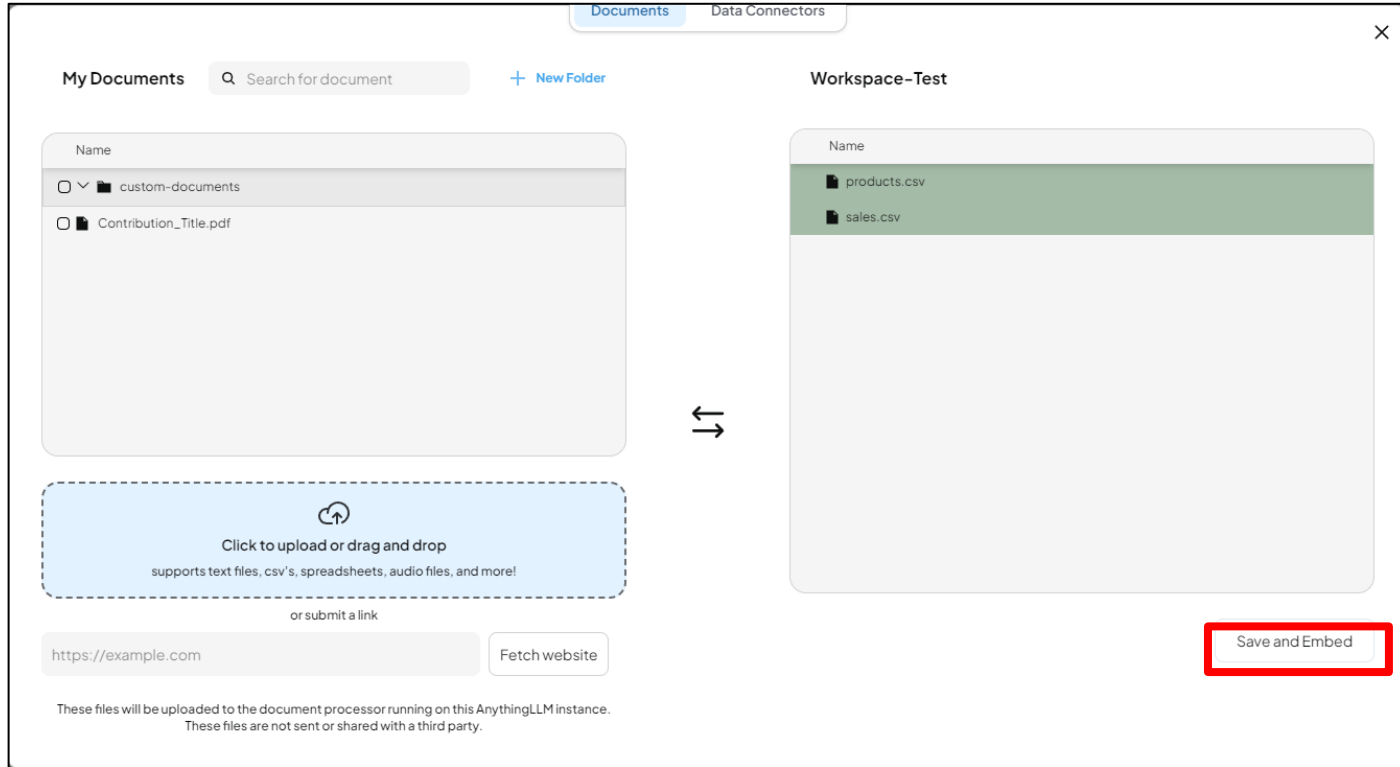


The screenshot illustrates the 'Anything LLM' interface for document management. On the left, a sidebar shows a 'Workspace-Test' button with an upload icon. A large white arrow points from this sidebar to the main 'My Documents' panel. This panel is divided into two sections: a document list and an upload area. The document list shows files like 'products.csv' and 'sales.csv', with a 'Move to Workspace' button highlighted by a red box and the number '3'. The upload area, highlighted by a red box and the number '1', prompts the user to 'Click to upload or drag and drop' and includes a 'Fetch website' button. A double-headed arrow connects the 'My Documents' panel to the 'Workspace-Test' panel on the right, which currently shows 'No Documents'. Red numbers '2' and '3' are placed near the document list and the 'Move to Workspace' button, respectively.

NEW THREAD

Source: Self-created

Retrieval-Augmented Generation



The screenshot displays the SAP UCC interface for document management. At the top, there are tabs for "Documents" and "Data Connectors". The "Documents" tab is active, showing a "My Documents" section with a search bar and a "New Folder" button. Below this, a list of documents is shown, including a folder named "custom-documents" and a file named "Contribution_Title.pdf". To the right, the "Workspace-Test" section is visible, showing a list of files: "products.csv" and "sales.csv". A double-headed arrow indicates a connection between the "My Documents" and "Workspace-Test" sections. Below the document list, there is a dashed box containing a cloud upload icon and the text "Click to upload or drag and drop supports text files, csv's, spreadsheets, audio files, and more!". Below this, there is a text input field with the placeholder "https://example.com" and a "Fetch website" button. At the bottom right, a "Save and Embed" button is highlighted with a red border. A small "X" icon is located in the top right corner of the interface.

Documents Data Connectors

My Documents Search for document + New Folder

Name

- custom-documents
- Contribution_Title.pdf

Workspace-Test

Name

- products.csv
- sales.csv

Click to upload or drag and drop
supports text files, csv's, spreadsheets, audio files, and more!

or submit a link

https://example.com Fetch website

Save and Embed

These files will be uploaded to the document processor running on this AnythingLLM instance.
These files are not sent or shared with a third party.

Source: Self-created

Retrieval-Augmented Generation

- Independent Task (after providing your LLM with products.csv and sales.csv), **test** your LLM and **find out**:
 - What products are available?
 - What is the most expensive product?
 - What is the cheapest product?
 - When was the first product sold?
 - When was the last product sold?
 - Which product was sold most often?
 - How many products are there?

- Was the LLM able to answer all of these questions? Are there question, which the LLM can not answer correctly?

Retrieval-Augmented Generation

General Settings Chat Settings Vector Database Agent Configuration

Workspace LLM Provider

The specific LLM provider & model that will be used for this workspace. By default, it uses the system LLM provider and settings.



Ollama

Run LLMs locally on your own machine.



Workspace Chat model

The specific chat model that will be used for this workspace. If empty, will use the system LLM preference.

granite4:micro



Chat mode

Chat

Query

Chat will provide answers with the LLM's general knowledge *and* document context that is found.

Chat History

The number of previous chats that will be included in the response's short-term memory. *Recommend 20.* Anything more than 45 is likely to lead to continuous chat failures depending on message size.

20

System Prompt

The prompt that will be used on this workspace. Define the context and instructions for the AI to generate a response. You should provide a carefully crafted prompt so the AI can generate a relevant and accurate response.

You can insert **prompt variables** like: `{time}` , `{date}` , `{datetime}` , *+4 more...*

[View History](#)

Given the following conversation, relevant context, and a follow up question, reply with an answer to the current question the user is asking. Return only your response to the question given the above information following the users instructions as needed.

Query mode refusal response

When in **query** mode, you may want to return a custom refusal response when no context is found.

There is no relevant information in this workspace to answer your query.

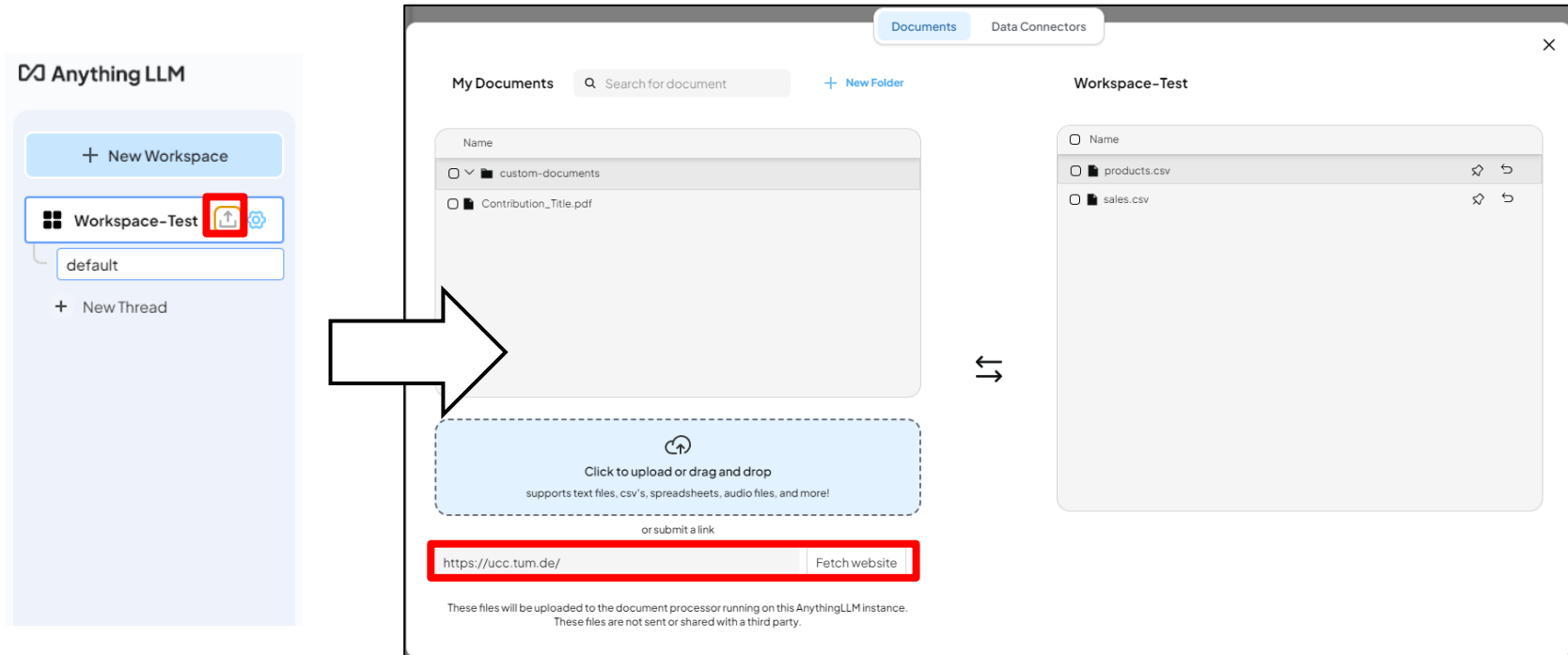
LLM Temperature

This setting controls how "creative" your LLM responses will be. The higher the number the more creative. For some models this can lead to incoherent responses when set too high.

Most LLMs have various acceptable ranges of valid values. Consult your LLM provider for that information.

0,1

Retrieval-Augmented Generation



The screenshot displays the 'Anything LLM' application interface. On the left sidebar, the 'Workspace-Test' workspace is highlighted with a red box around its icon. A large white arrow points from this workspace to the main content area. The main area is divided into two panels: 'My Documents' and 'Workspace-Test'. The 'My Documents' panel shows a list of documents: 'custom-documents' (expanded) and 'Contribution_Title.pdf'. Below this list is a dashed box with the text 'Click to upload or drag and drop' and 'supports text files, csv's, spreadsheets, audio files, and more!'. Below this is a text input field containing 'https://ucc.tum.de/' and a 'Fetch website' button, both highlighted with a red box. The 'Workspace-Test' panel shows a list of documents: 'products.csv' and 'sales.csv'. A double-headed arrow connects the two panels. At the bottom, a note states: 'These files will be uploaded to the document processor running on this AnythingLLM instance. These files are not sent or shared with a third party.'

NEW THREAD

Source: Self-created

Retrieval-Augmented Generation



What is the SAP University Competence Center?



The SAP University Competence Center (UCC) is a global network of universities that partner with SAP to provide students with a comprehensive education in business processes, IT, and SAP technologies. The UCCs offer a range of academic programs, research opportunities, and certifications that prepare students for careers in the SAP ecosystem.



Retrieval-Augmented Generation

- Independent Task:
 - Identify a topic that is unknown to the LLM. This means the LLM should not be able to provide reasonable or accurate answers to questions about this topic initially.
 - Explain the topic to the LLM. You can do this in one of two ways:
 - Share a URL that describes the topic in detail.
 - Create a document or file that explains the topic and upload it to your workspace.
 - In a new thread, ask the same questions again. Evaluate whether the LLM can now provide accurate and informed answers based on the explanation you provided earlier.
 - You can establish several workspaces connected to different LLMs

Agenda

- Recap
- Basics
- Deploying IBM Granite on Power10
- Zero-Shot Experiments
- Retrieval-Augmented Generation (RAG)
- **Function Calling (Agentic AI)**
- Visual Studio Code + Continue
- Autocomplete

Agentic AI Requirements

- **Tools:** AnythingLLM, MobaXterm
- **LLM model:** models/granite-4.0-micro-Q5_K_M.gguf
- Search API Key
- Chat mode: Query
- Chat History: 1
- Temperature: 0.5
- Average query response time:
- DuckDuckGO: 80-130s
- Tavily Search: 40-50s

Chat mode

Chat Query

Query will provide answers *only* if document context is found.

Chat History

The number of previous chats that will be included in the response's short-term memory. *Recommend 20.* Anything more than 45 is likely to lead to continuous chat failures depending on message size.

1

System Prompt

The prompt that will be used on this workspace. Define the context and instructions for the AI to generate a response. You should provide a carefully crafted prompt so the AI can generate a relevant and accurate response.

You can insert **prompt variables** like: {time}, {date}, {datetime}, [+2 more...](#)

[View History](#)

You are a web-browsing assistant specialized in retrieving up-to-date information. When searching, always prioritize official and trustworthy sources. Ignore results from unofficial blogs, forums, or unrelated websites. Answer the user's question clearly.

Clear

[Publish to Community Hub](#)

Query mode refusal response

When in **query** mode, you may want to return a custom refusal response when no context is found.

There is no relevant information in this workspace to answer your query.

LLM Temperature

This setting controls how "creative" your LLM responses will be. The higher the number the more creative. For some models this can lead to incoherent responses when set too high.

Most LLMs have various acceptable ranges of valid values. Consult your LLM provider for that information.

0.5

- Agentic AI is an [artificial intelligence](#) system that can accomplish a specific goal with limited supervision. It consists of AI agents—machine learning models that mimic human decision-making to solve problems in real time. [Link](#)

REAL-WORLD APPLICATIONS OF AI AGENTS



Customer Service
and Chat Support



Sales & Marketing
Automation



Finance & Banking



Healthcare



Manufacturing & IoT



Education

Source (Translated): [Link](#)

- Can our model perform web-browsing and return us the start dates for the new semester at TUM?



when does the new semester start at TUM 2025?



The information provided does not specify the exact date for when the new semester starts at TUM in 2025. To find this information, please visit the official TUM website or contact their admissions office directly.

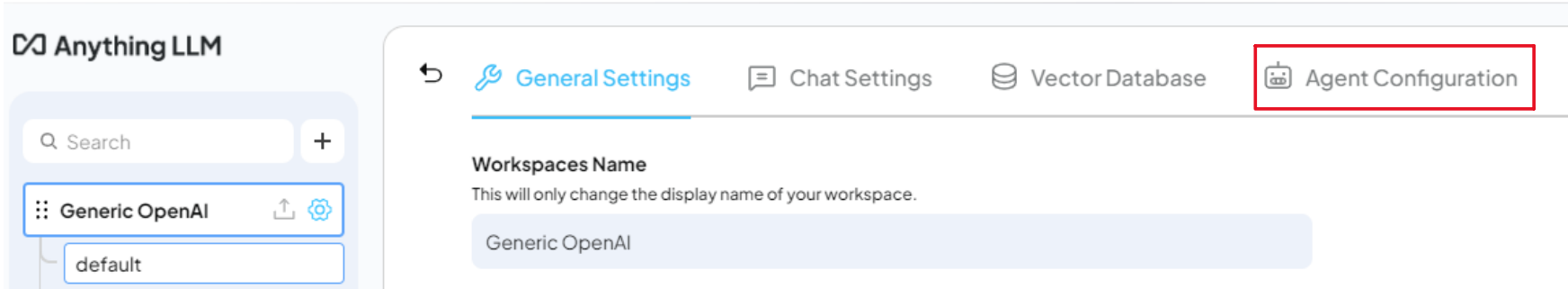


[Show citations >](#)

- We quickly realize our chat model only uses its training data and the data we embedded but it cannot search the internet to find results.

Agentic AI

- Let's configure our first agent.



- Go to the settings in your workspace and then select “Agent Configuration”

Agentic AI

  General Settings  Chat Settings  Vector Database  Agent Configuration



Performance of LLMs that do not explicitly support tool-calling is highly dependent on the model's capabilities and accuracy. Some abilities may be limited or non-functional.

Workspace Agent LLM Provider

The specific LLM provider & model that will be used for this workspace's @agent agent.



Generic OpenAI

Connect to any OpenAI-compatible service via a custom configuration



Multi-model support is not supported for this provider yet.

Agent's will use the model set for the workspace or the model set for the system.

Configure Agent Skills

Customize and enhance the default agent's capabilities by enabling or disabling specific skills. These settings will be applied across all workspaces.

Agentic AI

- Toggle on the Web Search agent and save the changes.

Agent Skills

RAG & long-term memory	On >
View & summarize documents	On >
Scrape websites	Off >
Generate & save files to browser	Off >
Generate charts	Off >
Web Search	Off >
SQL Connector	Off >

Live web search and browsing

☐



Enable your agent to search the web to answer your questions by connecting to a web-search (SERP) provider. Web search during agent sessions will not work until this is set up.

Agentic AI

- Choose a web search provider. 2 free options we recommend:
- 1) DuckDuckGO + easy setup, no API Key needed – slow responses
- 2) Tavily Search: + fast responses – with API Key

☰ Live web search and browsing



Enable your agent to search the web to answer your questions by connecting to a web-search (SERP) provider. Web search during agent sessions will not work until this is set up.



Please make a selection

Web search will be disabled until a provider and keys are provided.



☰ Live web search and browsing



Enable your agent to search the web to answer your questions by connecting to a web-search (SERP) provider. Web search during agent sessions will not work until this is set up.



Tavily Search

Tavily Search API. Offers a free tier with 1000 queries per month.



You can get an API key [from Tavily](#).

API Key

.....

Agentic AI

- Let's put it to the test: Click on @ to see the available agents

@agent - the default agent for this workspace.

rag-search web-scraping web-browsing save-file-to-browser list-documents
summarize-document chart-generation

custom agents are coming soon!

← Click here to
use the agent
functions

Send a message



Agentic AI



@agent web-browsing when does the new semester start at TUM 2025?



Agent @agent invoked. Swapping over to agent chat. Type /exit to exit agent execution loop ea...



The new semester at TUM (Technical University of Munich) typically starts in September. According to the search results, the university follows a semester system with two terms:

01. Fall Semester: Starts around September 20 and ends on December 20.
02. Winter Semester: Begins around January 15 and concludes on April 14.

For the year 2025, based on TUM's past patterns, the new semester would likely start in September 2025 (Fall Semester). However, please verify with the official TUM website or contact their admissions office for the most accurate and up-to-date information.

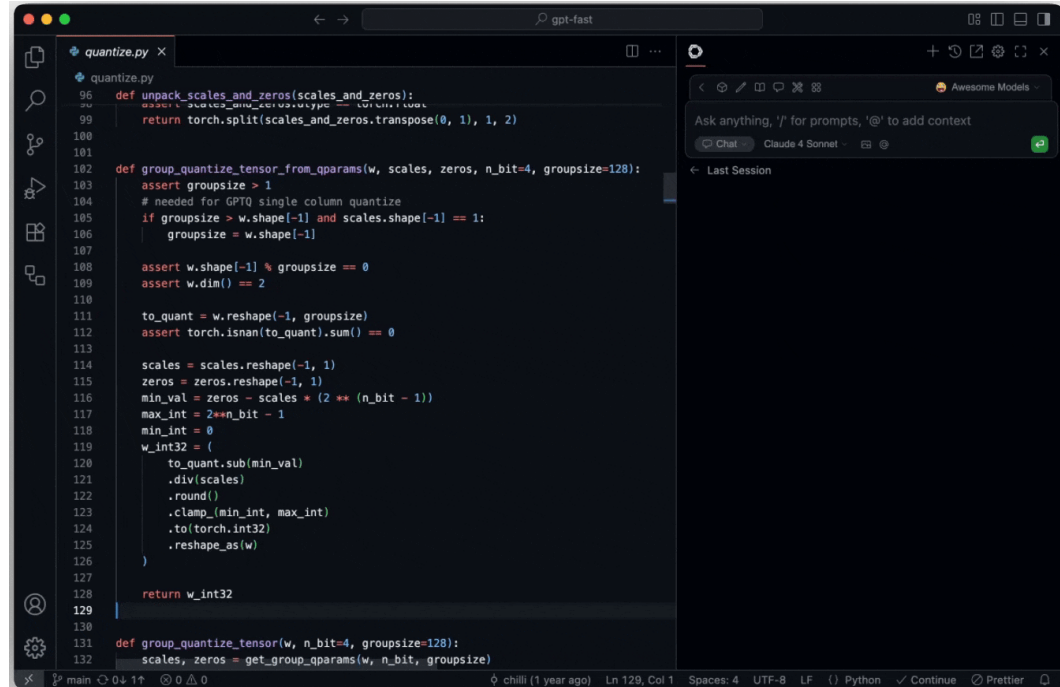
- Can our agent fetch dynamic information it does not already know about?
- Test the following queries on your own:
- 🔍 “What are the application deadlines for Bachelor programs at TUM in 2025?”
- 🔍 “Which events are scheduled at TUM Campus Garching in October 2025?”
- 🔍 “What are the current tuition fees for international students at TUM?”
- 🔍 “Who is the current president of TUM and what are their latest initiatives?”

Agenda

- Recap
- Basics
- Deploying IBM Granite on Power10
- Zero-Shot Experiments
- Retrieval-Augmented Generation (RAG)
- Function Calling (Agentic AI)
- **Visual Studio Code + Continue**
- Autocomplete

Visual Studio Code and Continue

- In this section we will connect our LLM with a code extension to help us with code generation.
- Continue is a type of add-on for Visual Studio Code that generates code suggestions and code for everyday programming.

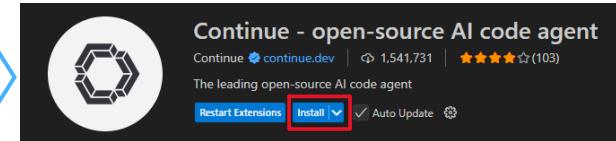
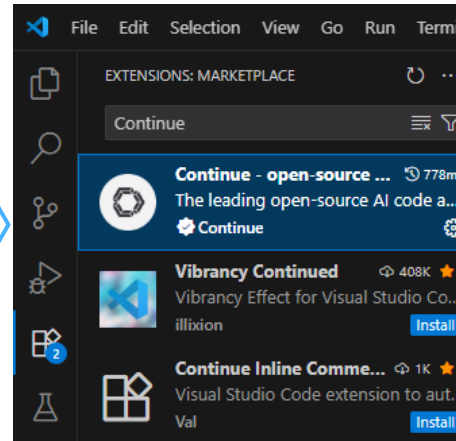
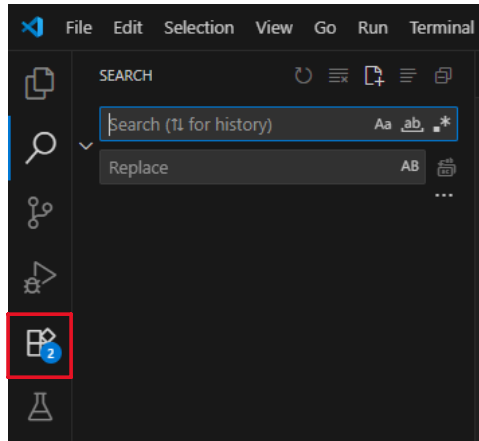


```
quantize.py
96 def unpack_scales_and_zeros(scales_and_zeros):
97     scales, zeros_and_zeros = torch.split(scales_and_zeros, [scales_and_zeros.shape[0], 1], 1)
98     return torch.split(scales_and_zeros.transpose(0, 1), 1, 2)
99
100
101
102 def group_quantize_tensor_from_qparams(w, scales, zeros, n_bit=4, groupsize=128):
103     assert groupsize > 1
104     # needed for GPTQ single column quantize
105     if groupsize > w.shape[-1] and scales.shape[-1] == 1:
106         groupsize = w.shape[-1]
107
108     assert w.shape[-1] % groupsize == 0
109     assert w.dim() == 2
110
111     to_quant = w.reshape(-1, groupsize)
112     assert torch.isnan(to_quant).sum() == 0
113
114     scales = scales.reshape(-1, 1)
115     zeros = zeros.reshape(-1, 1)
116     min_val = zeros - scales * (2 ** (n_bit - 1))
117     max_int = 2 ** n_bit - 1
118     min_int = 0
119     w_int32 = (
120         to_quant.sub(min_val)
121         .div(scales)
122         .round()
123         .clamp_(min_int, max_int)
124         .to(torch.int32)
125         .reshape_as(w)
126     )
127
128     return w_int32
129
130
131 def group_quantize_tensor(w, n_bit=4, groupsize=128):
132     scales, zeros = get_group_qparams(w, n_bit, groupsize)
```

[Source link](#)

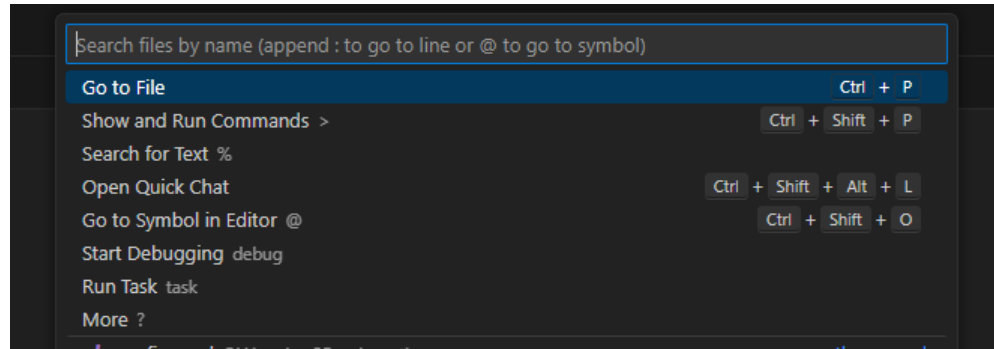
Visual Studio Code and Continue

1. Download Visual Studio Code Visual Studio Code - Code Editing. Redefined
2. Download the latest version of Python Download Python | Python.org
3. In VSCode, go to Extensions (or press Ctrl+Shift+x) and download “Continue - open-source AI code agent.”

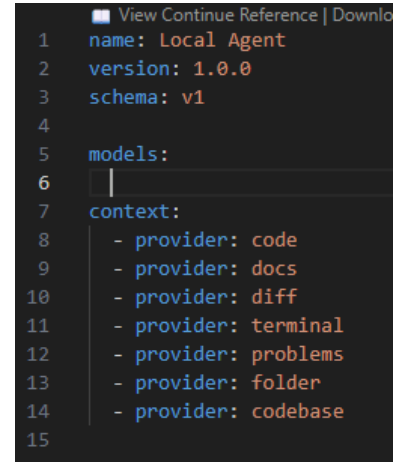


Visual Studio Code and Continue

4. Go to the search list and search for “config.yaml” → there you have to establish the connection to your Granite container.
5. In the “config.yaml” file in “models:” field, create a model to be used with “-”.



Edit this file



A screenshot of the "config.yaml" file in Visual Studio Code. The file contains the following YAML configuration:

```
1 name: Local Agent
2 version: 1.0.0
3 schema: v1
4
5 models:
6   |
7 context:
8   - provider: code
9   - provider: docs
10  - provider: diff
11  - provider: terminal
12  - provider: problems
13  - provider: folder
14  - provider: codebase
15
```

Continue Setup

6. For your model, define the following parameters: **name**, **model**, **provider**, **apiBase** with corresponding values
An example model looks like this:

models:

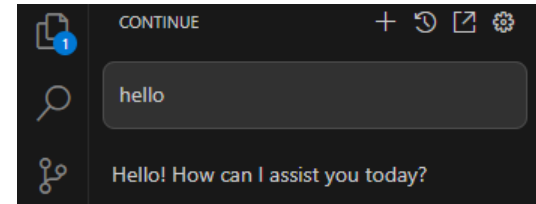
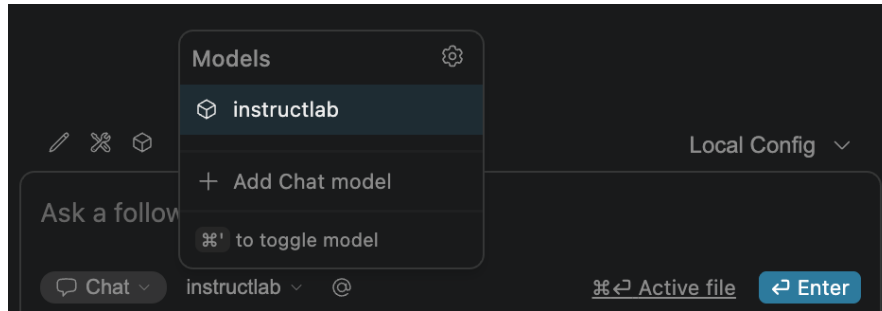
- **name**: <User defined name> # Choose something descriptive but short.
- model**: <Exact model identifier> # This must match the model's actual name as exposed by the API/container.
- provider**: <Backend provider type>
- apiBase**: <Base URL of the API endpoint> #<http://sose26dc.ucc.cit.tum.de>:<port>

```
! config.yaml X
Users > noahkim > .continue > ! config.yaml
  View Continue Reference | Download YAML extension for Intellisense
1  name: Local Config
2  version: 1.0.0
3
4  models:
5    - name: ollamatest
6      provider: ollama
7      model: granite4
8      roles:
9        - chat
10       - edit
11       - apply
12       - autocomplete
13      apiBase: http://granitelp1.ucc.cit.tum.de:5002
14
```

Adjust the apiBase accordingly!

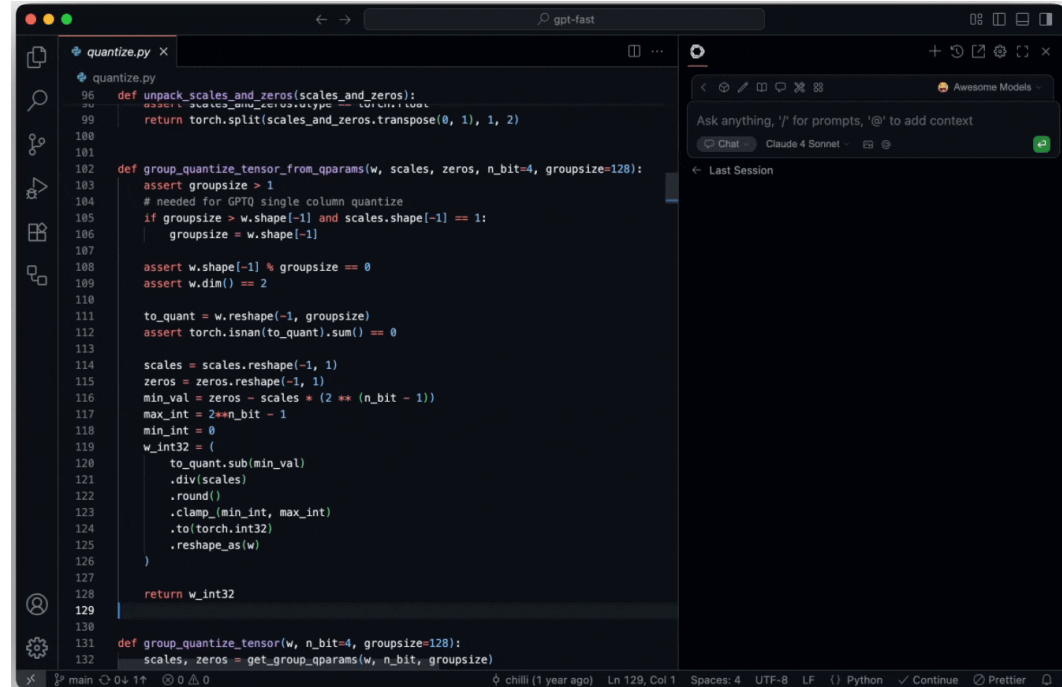
Continue Setup

- Let's try the chat function via Continue (CTRL + L)
- Select your LLM model
- We are ready for our test query:



Continue Autocomplete

In this section we will focus on what Continue Extension can do beyond the chat functionality.



```
quantize.py
96 def unpack_scales_and_zeros(scales_and_zeros):
97     scales, zeros = scales_and_zeros.type(torch.float)
98     return torch.split(scales_and_zeros.transpose(0, 1), 1, 2)
99
100
101
102 def group_quantize_tensor_from_qparams(w, scales, zeros, n_bit=4, groupsize=128):
103     assert groupsize > 1
104     # needed for GPTQ single column quantize
105     if groupsize > w.shape[-1] and scales.shape[-1] == 1:
106         groupsize = w.shape[-1]
107
108     assert w.shape[-1] % groupsize == 0
109     assert w.dim() == 2
110
111     to_quant = w.reshape(-1, groupsize)
112     assert torch.isnan(to_quant).sum() == 0
113
114     scales = scales.reshape(-1, 1)
115     zeros = zeros.reshape(-1, 1)
116     min_val = zeros - scales * (2 ** (n_bit - 1))
117     max_int = 2 ** n_bit - 1
118     min_int = 0
119     w_int32 = (
120         to_quant.sub(min_val)
121         .div(scales)
122         .round()
123         .clamp_(min_int, max_int)
124         .to(torch.int32)
125         .reshape_as(w)
126     )
127
128     return w_int32
129
130
131 def group_quantize_tensor(w, n_bit=4, groupsize=128):
132     scales, zeros = get_group_qparams(w, n_bit, groupsize)
```

gpt-fast

Awesome Models

Ask anything, '/' for prompts, '@' to add context

Chat Claude 4 Sonnet

Last Session

main 0 1 0

chilli (1 year ago) Ln 129, Col 1 Spaces: 4 UTF-8 LF Python Continue Prettier

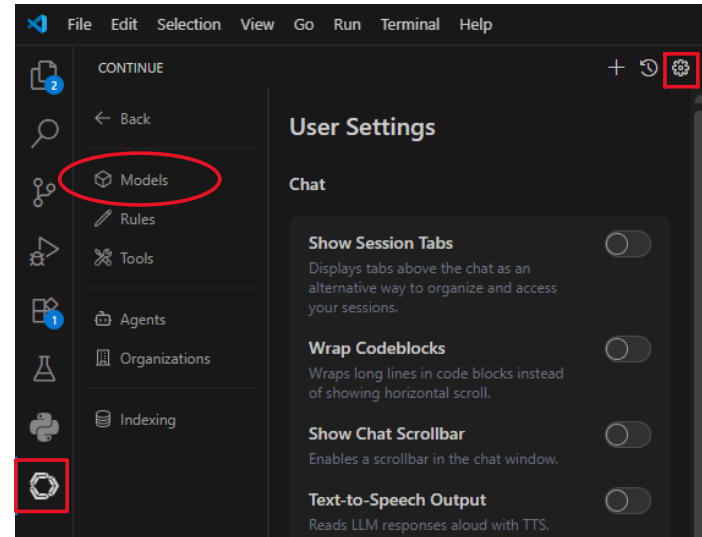
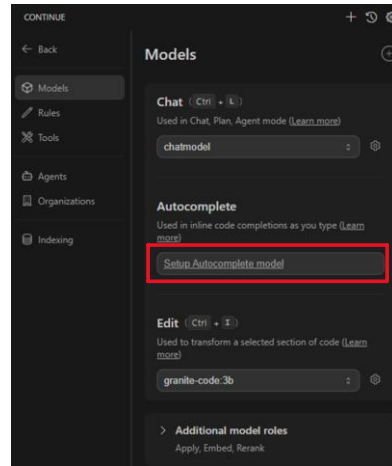
[Source link](#)

Autocomplete

- Autocomplete functionality enables inline code suggestions and keyboard shortcuts for accepting, rejecting, or partially accepting completions.
- Open VS Code and click on Continue icon on the left handlebar. Then open Continue settings and the models.

Here we have an overview of the models we use

We next want to configure our autocomplete model.



Autocomplete: Which LLM?

- Goal: Ultra-Low Latency (Sub-200ms)

Use Case	Model Examples	Typical Size	Common Quantization	What Matters Most	Why It Fits
Chatting	GPT-style, Claude, Mistral	7B – 70B+	Q5_K_M, Q8_0, FP16	Reasoning, long context, instruction-following	Larger models → better conversation quality
Autocomplete	Code Llama (small), StarCoder, Mistral 7B	1B – 7B	Q4_K_M, Q4_0	Low latency, fast token generation	Small + heavily quantized → instant suggestions
Coding	Code Llama, StarCoder, DeepSeek-Coder	7B – 34B	Q4_K_M, Q5_K_M, Q6_K	Code accuracy, structure, longer context	Code-trained models → better completions

Autocomplete: How to set Ollama?

- Problem: serving two or more LLMs in one ollama instance
- Depending on your RAM, model has to be loaded in and out
- Model has keep_alive timer
- See Ollama environment variables: `env | grep OLLAMA` (inside container)

OLLAMA_HOST	IP Address for the ollama server
OLLAMA_KEEP_ALIVE	The duration that models stay loaded in memory (default "5m")
OLLAMA_MAX_LOADED_MODELS	Maximum number of loaded models
OLLAMA_MAX_QUEUE	Maximum number of queued requests
OLLAMA_NUM_PARALLEL	Maximum number of parallel requests
OLLAMA_LOAD_TIMEOUT	How long to allow model loads to stall before giving up (default "5m")

Live Demo: Sanity Check

- Test if the LLM that we configured can interpret the function names and fill in the middle.

```
20 def sum(a, b, c):  
21     return a + b + c
```

```
20 def sum_of_squares(a, b, c):  
21     return a ** 2 + b ** 2 + c ** 2
```

```
def hello_world():  
    print("Hello, world!")
```

Live: Continue and Ollama

Task:

Try to chat with Coder-LLMs, what do you see?

Once you have a good LLM, define a method with a good naming convention and describe what the method does.

Hint: Avoid using zero-shot prompting and use Docstrings inside methods.

→ Few-shot prompting

```
5 def random_math_question():  
6     """  
7     asks a random math question and returns the answer.  
8     """
```

```
25 def random_math_question():  
26     """  
    eg. 1 + 2 =?  
    """  
    a, b = random.randint(0, 9), random.randint(0, 9)  
    answer = input(f"{a} + {b} = ")  
    return int(answer) == (a + b)
```

```
def phone_number_generator(country_code):  
    """  
    takes a country code as input and concatenates this with a valid 10 digit random number.  
    """  
    country_code = input()  
    phone_number = random.randint(10 ** (9 - country_code), 10 ** (10 - country_code))  
    return f"Your phone number is {phone_number}"
```

Task: “Code With Comments Only”

→ Complete the partially written methods below. **Type descriptive comments** to fill the missing code. **Inspect** the generated code, tweak it and **test the outputs**. Is the autocomplete correct?

```
def is_prime(n: int) -> bool:
    """Your comment goes here."""
    # TODO: implement

def fibonacci(n: int) -> list[int]:
    """Your comment goes here."""
    # TODO: implement
```

Bonus Task: Write a function that returns if a word is **palindrome** and test it.

Task: “Code With Comments Only”

def is_prime(n: int) -> bool:

"""Return True if n is a prime number."""

TODO: implement

def fibonacci(n: int) -> list[int]:

"""Return a list containing the first n Fibonacci numbers."""

TODO: implement

```
25 def is_prime(n: int) -> bool:
26     """Return True if n is a prime number."""
27     if n < 2:
28         return False
29     for i in range(2, int(n ** 0.5) + 1):
30         if n % i == 0:
31             return False
32     return True
33
34
35 def fibonacci(n: int) -> list[int]:
36     """Return a list containing the first n Fibonacci numbers."""
37     a, b = 0, 1
38     fibs = [a]
39     for i in range(n - 1):
40         a, b = b, a + b
41         fibs.append(b)
42     return fibs
43
```

```
def is_palindrom(text):
    """
    a word is a palindrom when it is written the same from the left and the right.
    This function takes a word and checks if the word is a palindrom and returns yes if it is.
    """
    return text==text[::-1]
```

Key Takeaways

- Containers allow running several Ollama instances
- Sizing the LPAR accordingly is essential for running LLMs in containers
- **Few-shots prompting** guides the model and delivers more predictable outputs.
- Different models for different purposes → always trade-off between latency and accuracy
- **Quantization** and **model size** affect speed. Smaller models (e.g., 3B) → faster autocomplete. Larger models (7B–15B) → better code reasoning, slower response.
- **Context size** matters: The larger the model's context window, the better it can understand surrounding text and continue it accurately.
- Autocomplete is **probabilistic** — always review model suggestions